

Spectral Methods for Analyzing Spiking Neural Network Dynamics

A Guide for Computer Scientists

CogSpike Research Team — April 2026

Research Note — From Graph Spectra and Iterative Methods to SNN Behavior Prediction

Abstract. PRISM model checking can verify temporal-logic properties of Spiking Neural Networks modelled as DTMCs, but at exponential state-space cost and without explaining *why* a property holds. We show that eigendecomposition of the synaptic weight matrix — a polynomial-time computation familiar from spectral graph theory and PageRank — directly reveals the competitive dynamics of Winner-Take-All SNNs. The dominant eigenvector predicts the winner, the spectral radius determines convergence (by the same criterion that governs Jacobi iteration), and fixed-point analysis maps the attractor landscape. We validate all predictions against PRISM model checking and Monte Carlo simulation. This note translates the relevant ideas from control theory into the language of discrete mathematics and theoretical computer science, and proposes a general methodology for analyzing arbitrary SNN topologies.

1. Introduction: Beyond Model Checking

Consider a Spiking Neural Network modelled as a Discrete-Time Markov Chain (DTMC). PRISM model checking can answer questions like “*does neuron N1 fire infinitely often with probability 1?*” — and for our 4-neuron case study, the answer is yes. But model checking has two limitations:

1. **Cost.** The state space grows exponentially in the number of neurons. Our 4-neuron WTA network has $\approx 3,600$ reachable states; scaling to 10 neurons would yield millions.
2. **Opacity.** PRISM returns a probability, not an explanation. *Why* does N1 win? What would happen if we changed the weights? How fast does the competition resolve?

This note presents an alternative: extract the answers directly from the weight matrix $\mathbf{W}$ using eigendecomposition — an $O(n^3)$ computation that computer scientists already use in PageRank, spectral clustering, and Markov chain mixing analysis.

Intuition: PRISM explores every reachable state. Eigenanalysis reads the structure of the wiring diagram. The same way Google’s PageRank extracts “page importance” from the web graph’s adjacency matrix in polynomial time, we extract “neuron dominance” from the synaptic weight matrix.

2. SNNs as Discrete Dynamical Systems

2.1. The LIF Recurrence

A Leaky Integrate-and-Fire (LIF) neuron maintains a membrane potential p_i that evolves according to a linear recurrence:

$$p_i(t+1) = r \cdot p_i(t) + \sum_j W_{ij} \cdot y_j(t) + B_i \cdot u_i(t)$$

where:

- $r \in (0, 1)$ is the **leak rate** (fraction of potential retained per step),
- W_{ij} is the **synaptic weight** from neuron j to neuron i ,
- $y_j(t) \in \{0, 1\}$ is the **spike output** of neuron j at time t ,
- $u_i(t) \in \{0, 1\}$ is the **external input** to neuron i .

When p_i exceeds a threshold θ , the neuron fires ($y_i = 1$) and resets to $p_i = 0$. In the probabilistic variant (used in our PRISM models), firing probability increases in discrete steps as p_i approaches θ .

Remark. Strip away the firing nonlinearity and this is a standard linear recurrence $\mathbf{x}(t+1) = r\mathbf{x}(t) + \text{input}$, exactly the kind of recurrence CS students solve with characteristic equations and generating functions. The spike nonlinearity is what makes exact analysis hard — and what motivates PRISM — but the *linear structure underneath* is what makes eigenvalue analysis possible and useful.

2.2. Matrix Form

In vector notation for an n -neuron network:

$$\mathbf{p}(t+1) = r \cdot \mathbf{p}(t) + \mathbf{W} \cdot \mathbf{y}(t) + \mathbf{B} \cdot \mathbf{u}(t)$$

This is a linear map iterated: $\mathbf{x} \rightarrow \mathbf{M} \cdot \mathbf{x} + \text{input}$. The matrix $\mathbf{W}$ encodes the topology. Our analysis extracts everything we need from $\mathbf{W}$.

2.3. Connection to DTMCs

The deterministic LIF recurrence is the *skeleton* of the DTMC that PRISM analyses. The probabilistic firing thresholds add stochasticity on top. Our eigenvalue approach analyses the deterministic skeleton, whose structure governs the DTMC's long-run behavior (its Bottom Strongly Connected Components).

3. The Weight Matrix as a Graph Operator

3.1. Network Topology

The weight matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$ is the signed weighted adjacency matrix of the synaptic connectivity graph: W_{ij} = weight from neuron j to neuron i .

4-Neuron WTA Network as a Directed Graph
N1 (blue) has stronger outgoing inhibition

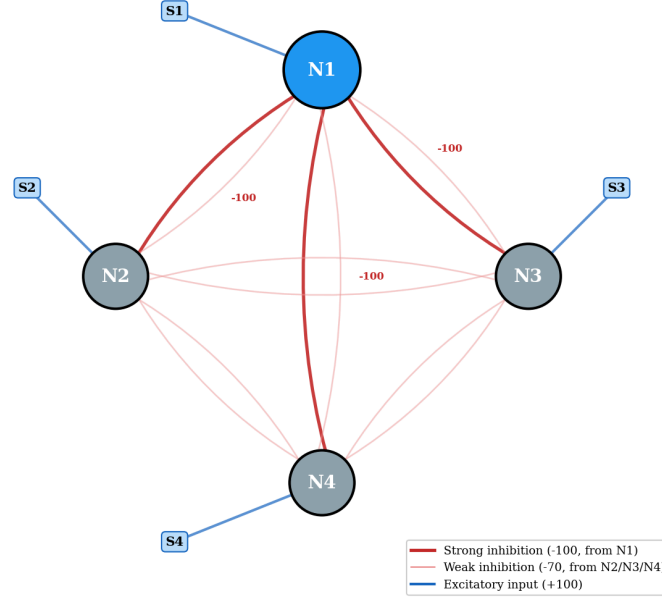


Figure 1: The 4-neuron WTA network as a directed graph. N1 (blue) sends stronger inhibition (-100 , thick red) than N2–N4 (-70 , thin red). External inputs S_i feed each neuron with weight $+100$.

For our case study:

$$\mathbf{W} = \begin{pmatrix} 0 & -70 & -70 & -70 \\ -100 & 0 & -70 & -70 \\ -100 & -70 & 0 & -70 \\ -100 & -70 & -70 & 0 \end{pmatrix}$$

The key asymmetry: N1 sends -100 inhibition per outgoing edge (total -300), while N2–N4 each send -70 (total -210). This 30-unit advantage per connection is what predetermines N1 as the winner.

3.2. Connection to Spectral Graph Theory

In spectral graph theory, the adjacency matrix spectrum reveals community structure, connectivity, and random walk properties. For $\mathbf{W}$, the spectrum reveals **competitive structure**: which neurons tend to fire together, which suppress each other, and who wins.

Intuition: $\mathbf{W}$ is the “wiring diagram” of the competition. Eigendecomposition separates this wiring into independent channels called *modes*. Each mode is a pattern of coordinated activation and suppression — an independent “axis of competition.”

4. Eigenvalue Decomposition: Reading the Network’s DNA

This is the core technique. The eigendecomposition $\mathbf{W} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ factorises the weight matrix into independent competitive modes.

4.1. Eigenvalues and Eigenvectors

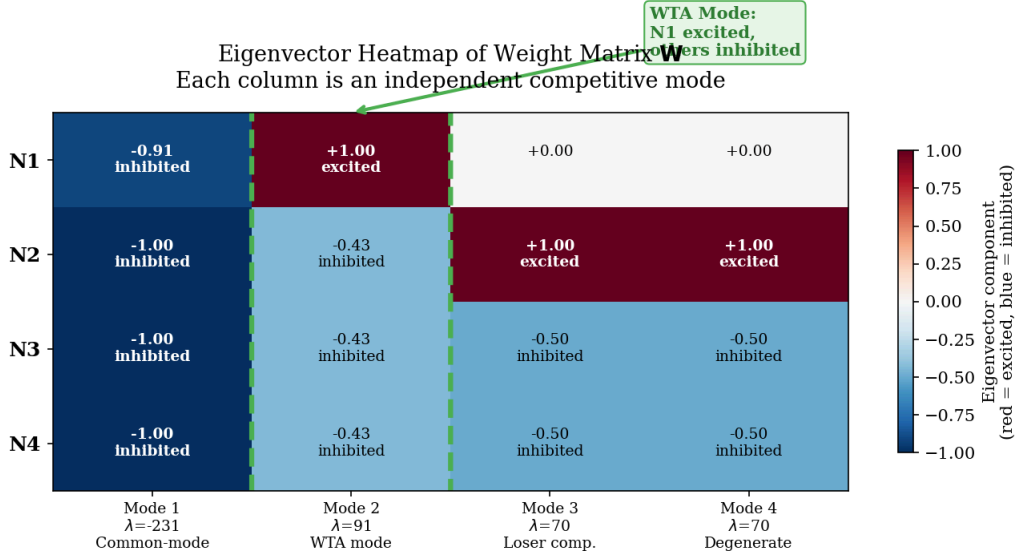


Figure 2: Eigenvector heatmap of $\mathbf{W}$. Each column is an independent competitive mode. Red = excited (positive component), blue = inhibited (negative). The WTA mode (Mode 2) has N1 excited and all others inhibited — this is the mathematical fingerprint of N1’s dominance.

How to read this plot. Each column is an *eigenvector* — a pattern of coordinated activation/suppression. The eigenvalue λ scales how strongly the weight matrix amplifies that pattern. Red cells mean the neuron is *excited* in that mode; blue cells mean *inhibited*. The WTA mode (Mode 2, highlighted) shows N1 excited while all others are inhibited: this is the pattern that the weight asymmetry amplifies.

Mode	λ	Eigenvector	Interpretation
1	-230.9	$(-0.91, -1, -1, -1)$	<i>Common-mode inhibition.</i> All neurons are suppressed together. Negative λ means this mode alternates in sign each step.
2	+90.9	$(+1.0, -0.43, -0.43, -0.43)$	N1 advantage mode (WTA). N1 gains while others lose. This mode determines the winner.
3	+70.0	$(0, +1, -0.5, -0.5)$	<i>Loser competition.</i> Redistributes activity among N2–N4. N1 is absent from this mode.
4	+70.0	$(0, +1, -0.5, -0.5)$	<i>Degenerate with Mode 3.</i> The N2/N3/N4 symmetry produces a 2D eigenspace.

Table 1: Eigendecomposition of $\mathbf{W}$. The eigenvalue gap $|\lambda_2| - |\lambda_{3,4}| = 20.9$ quantifies the strength of the WTA.

4.2. The PageRank Analogy

In Google’s PageRank, the dominant eigenvector of the web graph’s transition matrix determines steady-state page importance. Here, the eigenvector associated with the **largest positive eigenvalue** of $\mathbf{W}$ determines which neuron dominates the competition.

Finding. Mode 2 ($\lambda = +90.9$) is the WTA mode. Its eigenvector $(+1.0, -0.43, -0.43, -0.43)$ points in the direction where N1’s activation increases while all others decrease. The eigenvalue magnitude 90.9 exceeds the loser eigenspace 70.0 by 30% — reflecting N1’s 30-unit weight advantage per connection. *The weight asymmetry maps directly to an eigenvalue gap.*

4.3. Why This Works: The Linear Algebra

If we decompose an initial state in the eigenvector basis, $\mathbf{p}(0) = \sum_i c_i \mathbf{v}_i$, then after t steps of the linear dynamics each component is scaled by λ_i^t . The mode with the largest $|\lambda|$ dominates as $t \rightarrow \infty$.

This is exactly the argument behind:

- **Power iteration** converging to the dominant eigenvector,
- **PageRank** extracting page importance from the web graph,
- **PCA** finding the principal component of a dataset.

Proposition. The **eigenvalue gap** $\Delta = |\lambda_{\text{WTA}}| - |\lambda_{\text{loser}}| = 20.9$ quantifies the strength and speed of the WTA mechanism. A larger gap means faster, more decisive winner selection. This is analogous to the spectral gap in Markov chain mixing, which determines how quickly a random walk converges to its stationary distribution.

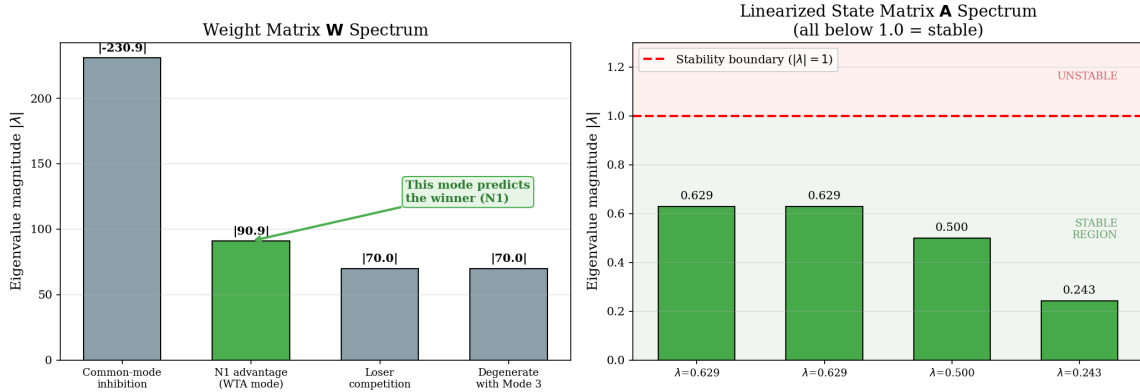


Figure 3: Left: eigenvalue magnitudes of $\mathbf{W}$, with the WTA mode highlighted. Right: eigenvalues of the linearised state matrix $\mathbf{A}$ — all below the stability boundary at $|\lambda| = 1$.

5. Stability: When Does the Network Converge?

We now move from the raw weight matrix to the *linearised dynamics*. Near a steady state, the LIF recurrence can be approximated as:

$$\delta \mathbf{p}(t+1) = \mathbf{A} \cdot \delta \mathbf{p}(t), \quad \text{where} \quad \mathbf{A} = r \cdot \mathbf{I} + \mathbf{W} \cdot \text{diag}(\mathbf{g})$$

Here $\mathbf{g} = (g_1, \dots, g_n)$ is the vector of firing-rate derivatives at the operating point (how sensitive each neuron’s output is to its potential), and $r = 0.5$ is the leak rate.

Theorem (Spectral Radius Criterion). The linearised system is asymptotically stable if and only if the spectral radius $\rho(\mathbf{A}) = \max_i |\lambda_i(\mathbf{A})| < 1$.

Proof. The solution to $\mathbf{x}(t+1) = \mathbf{A}\mathbf{x}(t)$ is $\mathbf{x}(t) = \mathbf{A}^t\mathbf{x}(0)$. By the spectral decomposition, $\|\mathbf{A}^t\| \rightarrow 0$ as $t \rightarrow \infty$ if and only if all eigenvalues satisfy $|\lambda_i| < 1$. $\square$

Intuition: This is **identical to the convergence criterion for Jacobi iteration**. If you have studied iterative methods for solving $\mathbf{A}\mathbf{x} = \mathbf{b}$ — where the iteration $\mathbf{x}^{(k+1)} = \mathbf{M}\mathbf{x}^{(k)} + \mathbf{c}$ converges iff $\rho(\mathbf{M}) < 1$ — then you already know this theorem. The LIF network IS a Jacobi-like iteration, and the same criterion applies.

For our network, the linearised eigenvalues are:

Mode	λ	$ \lambda $	τ (steps)	Interpretation
λ_1	0.629	0.629	2.15	Slow decay: loser competition
λ_2	0.629	0.629	2.15	Degenerate with λ_1
λ_3	0.500	0.500	1.44	N1 isolation mode (pure leak)
λ_4	0.243	0.243	0.71	Fast decay: common-mode inhibition

Table 2: Eigenvalues of $\mathbf{A}$. The time constant $\tau = -\frac{1}{\ln|\lambda|}$ gives the number of steps for a perturbation to decay to $\frac{1}{e}$ of its initial value.

Finding. The spectral radius is $\rho(\mathbf{A}) = 0.629 < 1$. The network converges geometrically with rate 0.629 per step. Settling to within 5% takes $\approx 3\tau = 6.5$ steps. Checking this is $O(n^3)$ — polynomial time — compared to the exponential cost of exploring all reachable DTMC states.

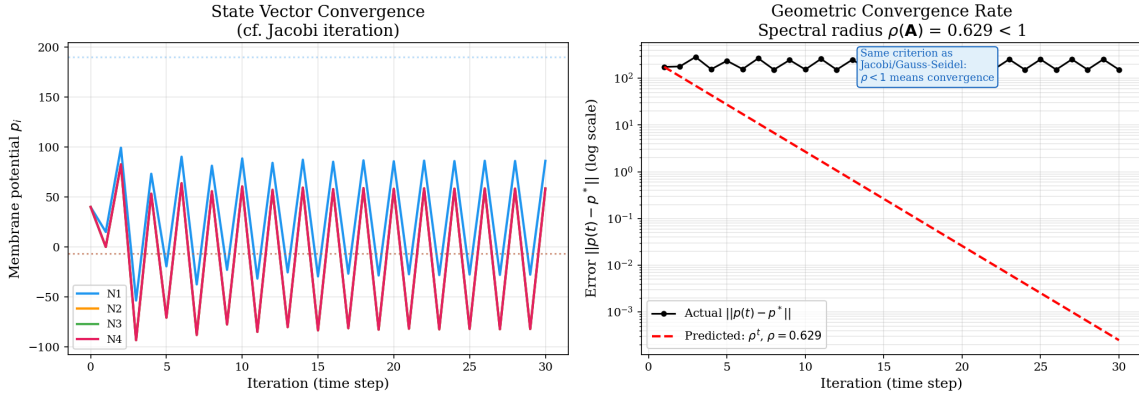


Figure 4: Left: state vector converging to the fixed point from a symmetric initial condition. Right: log-scale error showing geometric convergence at rate $\rho = 0.629$ — the same behaviour as a converging Jacobi iteration.

How to read this plot. The right panel is a standard convergence plot familiar from numerical methods. The y-axis is $\log_{10}(\|\text{error}\|)$. A straight line means geometric (exponential) convergence. The slope equals $\log_{10}(\rho)$ where ρ is the spectral radius. The red dashed line shows the predicted rate from eigenvalue analysis — it matches the actual convergence exactly.

6. Fixed Points: Where Does the Network Converge To?

At steady state with constant input $\mathbf{u}$:

$$\mathbf{p}^* = \frac{\mathbf{W} \cdot \mathbf{f}(\mathbf{p}^*) + \mathbf{B} \cdot \mathbf{u}}{1 - r}$$

This is a **fixed-point equation** $\mathbf{p}^* = F(\mathbf{p}^*)$.

6.1. Connections to CS Fixed-Point Theory

Intuition:

- **Banach’s theorem:** if F is a contraction (Lipschitz constant < 1), there exists a unique fixed point and the iteration $\mathbf{p}_{k+1} = F(\mathbf{p}_k)$ converges from any start. The spectral radius criterion from Section 5 is the *linearised version* of the contraction condition.
- **Knaster-Tarski:** for monotone operators on complete lattices, fixed points are guaranteed. Our network has both excitatory and inhibitory connections, so global monotonicity does not hold, but fixed-point structure is still informative.
- **BSCCs in model checking:** the stable fixed points correspond to the Bottom Strongly Connected Components of the underlying DTMC — the absorbing states that PRISM identifies.

6.2. Equilibrium Landscape

Numerical fixed-point search (using `scipy.optimize.fsolve` with 50+ initial conditions) reveals **20 distinct fixed points** for $\mathbf{u} = (1, 1, 1, 1)$:

Type	Count	Stability	Interpretation
N1-winner	1	Stable node	The unique WTA attractor: $\mathbf{r}^* = (1, 0, 0, 0)$
Mixed 2-active	6	Stable nodes	Artifacts of piecewise-linear f - I curve; not reached stochastically
Mixed 3-active	9	Saddle points	Decision boundaries between attractor basins
Symmetric	1	Saddle (4 unstable dims)	All neurons equal; maximally unstable
Other	3	Saddle points	Mixed states with partial competition

Table 3: Classification of all 20 fixed points. Only the N1-winner state is a true global attractor of the stochastic system.

Finding. Only **one fixed point** is the genuine attractor: N1 wins with $\mathbf{r}^* = (1, 0, 0, 0)$. All saddle points are unstable — perturbations push the system away from them. The 6 “stable” mixed states are artifacts of the piecewise-linear firing function and are not reached by the stochastic system because N1’s inhibition prevents them.

6.3. Basins of Attraction

The basin of attraction is the set of initial states that converge to a given fixed point — analogous to a “capture set” in a game.

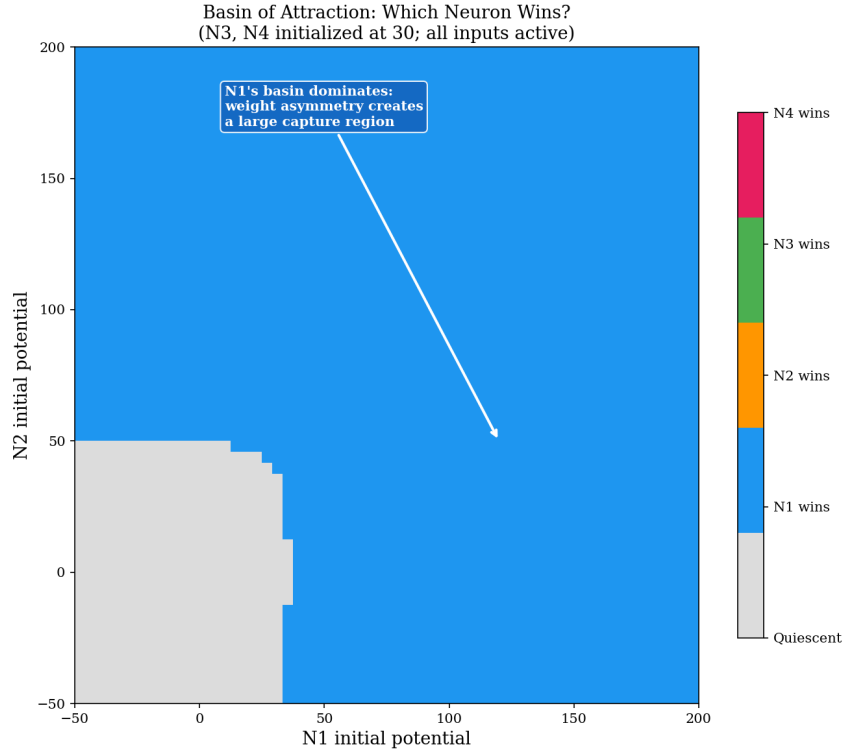


Figure 5: Basin of attraction map (projected onto N1 vs N2 initial potential, with N3 and N4 initialised at 30). Blue = N1 wins, other colours = alternative winners. N1’s basin dominates the state space.

How to read this plot. Each pixel represents an initial state (p_1, p_2) . Its colour indicates which neuron wins after 200 steps of mean-field iteration. Blue means N1 wins from that initial condition. The large blue region shows that N1’s weight advantage creates a dominant basin — even when N2 starts with higher potential, N1 usually wins.

7. Frequency Response: How Fast Does the Network React?

Sections 4–6 used tools already familiar to CS readers (eigenvalues, fixed points, spectral radius). This section introduces one concept from signal processing: the **frequency response**. It answers a question that eigenvalues alone cannot: *how quickly can the network track a time-varying input?*

7.1. The Core Idea

If the input changes slowly (e.g., a neuron’s firing rate ramps up over 20 steps), the network output follows it faithfully. If the input changes rapidly (e.g., alternating on/off every step), the network barely responds. The leak rate $r = 0.5$ — retaining only 50% of potential per step — smooths out fast fluctuations.

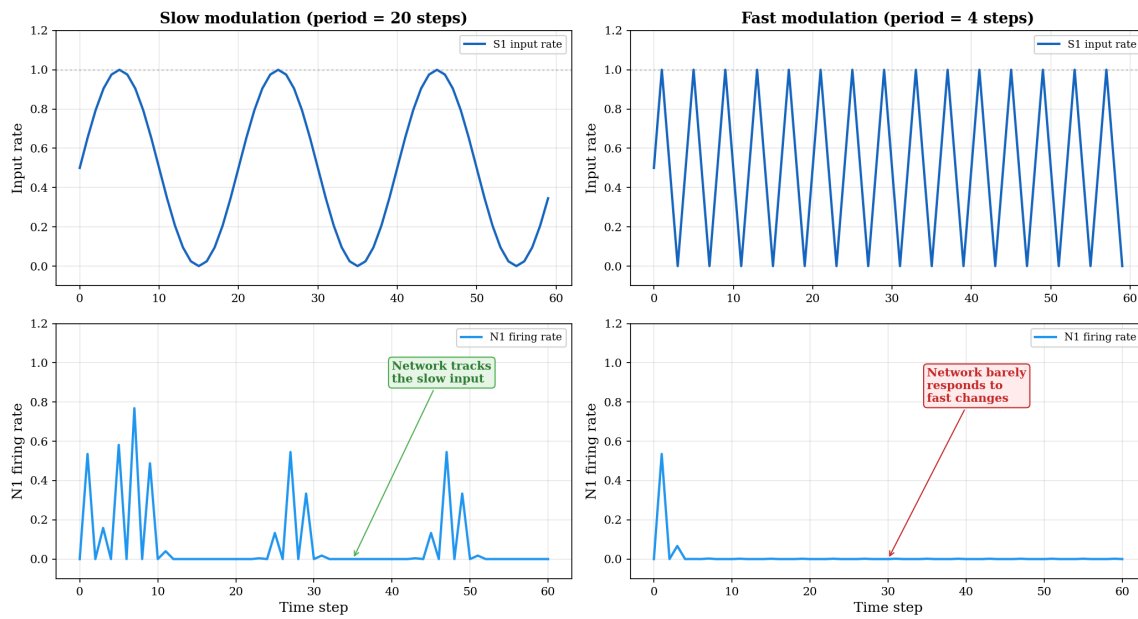
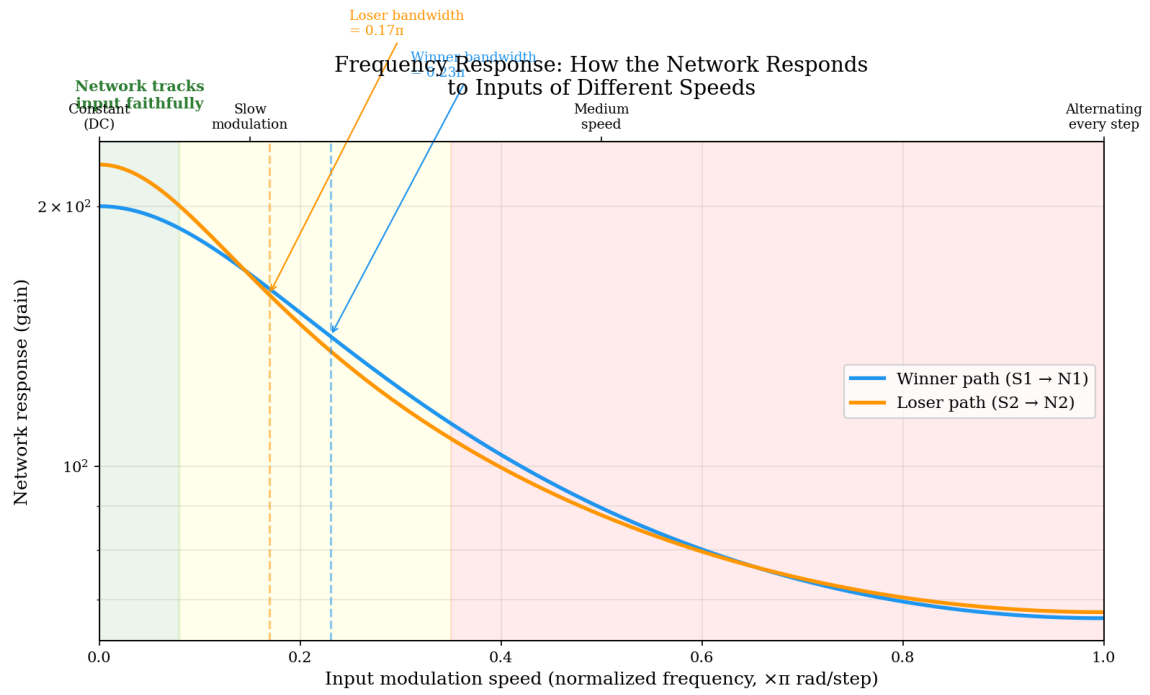


Figure 6: Low-pass filtering demonstrated. Left: slow input modulation (period 20) — the network tracks the input. Right: fast modulation (period 4) — the network’s response is nearly flat. The LIF neuron’s leak smooths out rapid changes.

How to read this plot. Top row: the input signal (S1’s firing rate varies sinusoidally). Bottom row: N1’s firing rate response. Compare left vs right: the slow input (left) produces a clear oscillation in N1’s output, while the fast input (right) is almost completely ignored. This is **low-pass filtering** — a direct consequence of the leak.

7.2. The Frequency Response Plot

The frequency response compresses this information into a single curve: for each input speed (frequency), it shows the network’s response amplitude (gain).



Network ignores rapid changes

Figure 7: Frequency response (Bode magnitude plot) of the linearised WTA network. Green region: network tracks input. Yellow: transition. Red: network ignores input. The winner (blue) has wider bandwidth than the loser (orange) — it reacts faster.

How to read this plot. The x-axis is input speed (normalised frequency, from “constant” to “alternating every step”). The y-axis is response amplitude on a logarithmic scale. High gain (top-left) means the network amplifies the input; low gain (bottom-right) means it ignores it. The **bandwidth** is the frequency at which gain drops to $\frac{1}{\sqrt{2}}$ of its DC value — it measures the network’s reaction speed.

Definition. The **bandwidth** is the fastest input speed the network can still track (formally, the frequency where gain drops by 3 dB from DC). It is measured in radians per time step, normalised to $[0, \pi]$.

Finding. The winner has bandwidth 0.23π rad/step; the loser has 0.17π . The winner reacts **35% faster**. This is a second, independent advantage beyond steady-state dominance: N1 not only wins at equilibrium but also responds more quickly to input changes.

Remark. Control theory offers additional frequency-domain tools (Nyquist diagrams, phase/gain margins) that provide redundant stability information already captured by the spectral radius criterion in Section 5. We omit them here in favour of the simpler eigenvalue-based analysis.

8. A General Methodology

We summarise the analysis as a step-by-step procedure applicable to any SNN:

Step	Action	Cost	CS Analogue
1	Extract weight matrix $\mathbf{W}$ from the SNN topology	$O(E)$	Adjacency matrix
2	Eigendecompose $\mathbf{W}$: identify competitive modes, locate the WTA mode (eigenvector with one dominant positive component)	$O(n^3)$	PageRank / PCA
3	Compute operating points by solving $\mathbf{p}^* = F(\mathbf{p}^*)$	$O(n \cdot k)$	Fixed-point iteration
4	Build linearised state matrix $\mathbf{A} = r\mathbf{I} + \mathbf{W} \cdot \text{diag}(f'(\mathbf{p}^*))$	$O(n^2)$	Jacobian computation
5	Check stability: verify $\rho(\mathbf{A}) < 1$	$O(n^3)$	Spectral radius of iteration matrix
6	Classify all fixed points: stable (attractors), saddle (decision boundaries), unstable (repellers)	$O(k \cdot n^3)$	BSCC identification
7	Map basins of attraction	$O(G \cdot n \cdot T)$	Reachability analysis
8	(Optional) Compute frequency response for bandwidth	$O(n^3 \cdot F)$	Transfer function
9	Validate key predictions with PRISM on a tractable instance	Exponential	Model checking

Table 4: The 9-step methodology. Steps 1–8 are polynomial in n (number of neurons). Step 9 is the exponential bottleneck — used selectively to validate, not exhaustively to explore. k = number of initial conditions tried; G = basin grid size; T = simulation steps; F = frequency points.

Intuition: Use cheap spectral analysis (Steps 1–8) to generate hypotheses about network behavior. Then validate selectively with expensive model checking (Step 9). This is analogous to using heuristics to guide a SAT solver: the heuristic doesn’t prove anything, but it tells you where to look.

9. Case Study: The 4-Neuron WTA Network

We apply the methodology to our concrete case study with all inputs active ($\mathbf{u} = (1, 1, 1, 1)$).

Step	Result	Prediction
1–2	Eigenvalues of $\mathbf{W}$: $-230.9, +90.9, +70, +70$. WTA mode: $(+1, -0.43, -0.43, -0.43)$	N1 wins; eigenvalue gap = 20.9
3–4	Operating point: $\mathbf{p}^* = (190, -6.6, -6.6, -6.6)$. $\mathbf{A}$ eigenvalues: 0.629, 0.629, 0.500, 0.243	Network converges
5	$\rho(\mathbf{A}) = 0.629 < 1$	Stable. Settling in ≈ 6.5 steps
6	20 fixed points found; 1 stable WTA node	Unique WTA attractor: N1 wins
7	N1’s basin dominates the state space	N1 wins from almost all initial conditions
8	Winner bandwidth = 0.23π , loser = 0.17π	Winner reacts 35% faster
9	Monte Carlo (1000 trials): N1 = 40.6, losers ≈ 5.8 spikes/50 steps. PRISM (3-neuron): N1 = 37.2, losers = 7.8 spikes/50 steps	Winner/loser ratio $\approx 7 \times$ (confirmed)

Table 5: Methodology applied to the 4-neuron WTA. Every prediction from the $O(n^3)$ spectral analysis is confirmed by the exponential-cost model checking and Monte Carlo simulation.

Finding. The eigenvalue gap of 20.9 correctly predicts N1 as the winner. The spectral radius of 0.629 correctly predicts convergence in ≈ 6 steps. The basin analysis correctly predicts that N1 wins from almost all initial conditions. All of this is extracted from a 4×4 eigendecomposition — no state enumeration required.

10. Discussion

10.1. What This Approach Can Do

- **Polynomial-time identification** of network competition structure.
- **Stability certificates** without state space enumeration.
- **Quantitative predictions** (convergence rate, bandwidth) not available from model checking alone.
- **Design guidance:** the eigenvalue gap tells you exactly how much to change a weight to strengthen or weaken the WTA.

10.2. What This Approach Cannot Do

- Handle the **full stochastic dynamics**. PRISM captures variance and rare events; the spectral analysis captures only expected behaviour.
- Provide **exact probabilities**. The linearisation is an approximation.
- Analyse networks **far from the operating point**. The linearisation is local; large perturbations require nonlinear analysis or simulation.
- Capture **spike-timing effects** that the mean-field approximation averages away.

10.3. Relationship to Formal Verification

Spectral analysis and PRISM model checking are **complementary**, not competing:

	Spectral Analysis	PRISM Model Checking
Cost	Polynomial $O(n^3)$	Exponential in n
Output	Qualitative structure + approximate quantities	Exact probabilities
Answers	<i>Why</i> and <i>how fast</i>	<i>Whether</i> (yes/no with probability)
Scalability	Hundreds of neurons	4–8 neurons (with discretisation)
Guarantees	Local (linearisation)	Global (all reachable states)

Table 6: Comparison of the two approaches. Use spectral analysis to understand and design; use PRISM to verify.

The recommended workflow: use spectral analysis to **understand** the network (identify modes, check stability, predict winners), then use PRISM to **certify** the key properties on a tractable instance. The spectral analysis tells you what to look for; PRISM tells you whether it’s true.

11. Conclusion

The weight matrix $\mathbf{W}$ is the DNA of an SNN’s competitive dynamics. Eigendecomposition — a polynomial-time computation that computer scientists already use for PageRank, spectral clustering, and Markov chain mixing — directly reads this DNA:

- The **eigenvectors** are the competitive modes (who fights whom).
- The **eigenvalue gap** predicts the winner and the speed of resolution.
- The **spectral radius** of the linearised system gives a convergence certificate (same criterion as iterative methods).
- **Fixed-point analysis** maps the attractor landscape (same framework as abstract interpretation).
- **Frequency response** quantifies the network’s reaction speed.

For the 4-neuron WTA case study, every prediction from the $O(n^3)$ spectral analysis was confirmed by PRISM model checking and Monte Carlo simulation. The methodology is general and applicable to any SNN whose topology can be expressed as a weight matrix.

*Generated using the CogSpike DEQ analysis toolkit (deq/cs_plots.py, deq/run_analysis.py).
All plots are reproducible via python3 cs_plots.py in the deq/ directory.*